Word2Vec

Word2Vec is a word embedding technique in natural language processing (NLP) that allows words to be represented as vectors in a continuous vector space. Researchers at Google developed word2Vec that maps words to high-dimensional vectors to capture the semantic relationships between words. It works on the principle that words with similar meanings should have similar vector representations. Word2Vec utilizes two architectures:

Word2Vec is a **neural network-based technique** developed by Google (Mikolov et al., 2013) to represent words as **dense vectors (embeddings)** in such a way that words with similar meaning or context are located close together in the vector space.

Key Ideas

1.Word Representation

1. Unlike one-hot encoding (sparse, high-dimensional, no semantic info), Word2Vec learns **low-dimensional, continuous vectors**.
2. Captures **semantic similarity** (e.g., "king" – "man" + "woman" ≈ "queen").

2.Training Objectives

Word2Vec is based on predicting word-context relationships from large corpora:

1. **CBOW (Continuous Bag of Words)** → predicts a target word from surrounding context words.
2. **Skip-Gram** → predicts surrounding context words given a target word.
    1. Works well for small datasets and rare words.

3.Architecture

1. Uses a **shallow neural network** (just one hidden layer).
2. Learns embeddings during training by optimizing prediction accuracy.

4.Optimization Techniques

Since vocabulary can be huge, Word2Vec uses:

1. **Negative Sampling** (predict a few negative samples instead of full softmax).
2. **Hierarchical Softmax** (efficient tree-based approximation).

Properties

- **Vector Arithmetic:**
  Captures relationships like
  `king − man + woman ≈ queen`
  `Paris − France + Italy ≈ Rome`
- **Semantic & Syntactic Similarity:**
  Words appearing in similar contexts get similar vectors.
- **Dimension Size:** Typically 100–300 dimensions.

One-Hot Representation of Words

Word	Representation
dog	[1, 0, 0, 0]
cat	[0, 1, 0, 0]
apple	[0, 0, 1, 0]
mango	[0, 0, 0, 1]

Steps to Create One-Hot Representation

1.Build Vocabulary

1. Collect all unique words in your text corpus.
2. Example sentence:
   “I like cats and dogs”
   Vocabulary = {I, like, cats, and, dogs}
   → Size = 5

2.Assign an Index to Each Word

1. I → 0
2. like → 1
3. cats → 2
4. and → 3
5. dogs → 4

3.Create Vectors

1. Each word is represented by a vector of length = vocabulary size.
2. All elements are **0**, except at the word’s index, which is **1**.

Example Representation

Vocabulary size = 5

Word	One-Hot Vector
I	[1, 0, 0, 0, 0]
like	[0, 1, 0, 0, 0]
cats	[0, 0, 1, 0, 0]
and	[0, 0, 0, 1, 0]
dogs	[0, 0, 0, 0, 1]

Pros:

•Simple, unique representation.

- **Cons:**

•Very high-dimensional (large vocab = long vectors).

•**No semantic similarity** (cat and dog are unrelated).

•Leads to sparsity → inefficient.

. Distributed Representation of Words

•Instead of sparse one-hot, each word is represented by a **dense low-dimensional vector** (e.g., 100–300 dimensions).

•Learned from context (words used in similar contexts have similar vectors).

•Example:

- dog → [0.21, -0.43, 0.65, …]
- cat → [0.25, -0.40, 0.63, …]

- **Pros:**

•Captures semantic similarity (dog and cat closer than dog and mango).

•Compact and computationally efficient.

•Basis for **Word2Vec, GloVe, FastText**.

SVD for Learning Word Representations (Matrix Factorization Approach)

Before Word2Vec, one approach was to use **co-occurrence matrices** + **matrix factorization**.

Step 1: Build Co-occurrence Matrix

- Suppose vocabulary = {dog, cat, apple, mango}
- Define a context window (say size = 2).
- Count how often each word appears near others → gives a matrix:

Target \ Context	dog	cat	apple	mango
dog	0	1	0	0
cat	1	0	1	0
apple	0	1	0	1
mango	0	0	1	0

Step 2: Apply SVD (Singular Value Decomposition)

We decompose the matrix:

$$X = U\Sigma V^T$$

- U :word vectors
- Σ: singular values (importance of features)
- V^T :context vectors

By keeping only top **k singular values** (dimensionality reduction), we get **dense embeddings** that capture major semantic patterns.

Step 1: What is a Co-occurrence Matrix?

Think of words like **friends sitting in a classroom**.

- If *dog* always sits near *cat*, they must be close friends.
- If *apple* always sits near *mango*, they're friends too.
- If they rarely sit together, they're not close.

👉 The co-occurrence matrix is just a **friendship table** that says *"how many times each word is seen near another word."*

Step 2: Vocabulary

We are given:

$$\text{Vocabulary} = \{dog, cat, apple, mango\}$$

So our matrix will be **4 × 4** (because each word can co-occur with every other word).

Rows = target word

Columns = context word

Step 3: Define Context Window

We say **window size = 2**.

👉 This means:

When looking at a word, we check the **2 words before** and **2 words after** it in a sentence.

Example sentence:

“dog and cat eat apple and mango”

- For the word **dog**, its context (within 2 words) = {and, cat}
- For **cat**, context = {dog, and, eat}
- For **apple**, context = {eat, and, mango}

…and so on.

Step 4: Build Co-occurrence Counts

Now we count how often each pair appears **together within the window**.

Let’s keep things simple with this tiny text:

Corpus = “dog chase cat and cat eat apple and mango”

Now, sliding window (size 2):

- **dog** → context = {chase, cat}
- **cat (first)** → context = {dog, chase, and}
- **cat (second)** → context = {and, eat, apple}
- **apple** → context = {cat, eat, and, mango}
- **mango** → context = {apple, and}

Step 5: Fill the Matrix
Now we count word-by-word:

Target \ Context	dog	cat	apple	mango
dog	0	1	0	0
cat	1	0	1	0
apple	0	1	0	1
mango	0	0	1	0

Explanation:
- **dog–cat = 1** (they appeared together once)
- **cat–dog = 1** (symmetric)
- **cat–apple = 1** (they co-occurred)
- **apple–mango = 1**

…and so on.
This is the **co-occurrence matrix**.

Step 1: Recall the SVD formula

SVD says:

$$A = U \Sigma V^T$$

Where:

- U = left singular vectors (for rows)
- Σ = singular values (diagonal)
- V^T = right singular vectors (for columns)

Mathematical approach:

1. Compute $\mathbf{A^TA}$ → to find **V** (columns).
2. Compute $\mathbf{AA^T}$ → to find **U** (rows).
3. Take **square roots of eigenvalues** → singular values (Σ).
4. Normalize eigenvectors → columns of U and V.

Step 2: Compute $\mathbf{A^TA}$

$$A^T A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}^T \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 3 & 0 & 1 \\ 1 & 0 & 2 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}$$

- $\mathbf{A^TA}$ is symmetric → eigenvectors will give **V** (columns).

Step 3: Compute eigenvalues of A^TA

Eigenvalues λ satisfy:

After solving the characteristic equation (algebraic calculation is long), the eigenvalues are approximately:

$$\lambda_1 \approx 5.828, \lambda_2 \approx 2, \lambda_3 \approx 1, \lambda_4 \approx 0.172$$

Step 4: Compute singular values Σ

Singular values = square roots of eigenvalues:

$$\sigma_i = \sqrt{\lambda_i}$$

$$\Sigma = \begin{bmatrix} \sqrt{5.828} & 0 & 0 & 0 \\ 0 & \sqrt{2} & 0 & 0 \\ 0 & 0 & \sqrt{1} & 0 \\ 0 & 0 & 0 & \sqrt{0.172} \end{bmatrix} \approx \begin{bmatrix} 2.414 & 0 & 0 & 0 \\ 0 & 1.414 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0.415 \end{bmatrix}$$

✅ These are exact **singular values** of A.

Step 5: Compute V (right singular vectors)

Solve $(A^T A)v = \lambda v$ for each eigenvalue λ → get **eigenvectors**.

Approximate V:

$$V \approx \begin{bmatrix} 0.354 & 0.5 & 0.707 & 0 \\ 0.5 & -0.707 & 0 & 0.5 \\ 0.5 & 0.5 & -0.707 & 0 \\ 0.5 & 0.5 & 0 & -0.707 \end{bmatrix}$$

- Columns of V = **right singular vectors**
- Then $\mathbf{V^T}$ is just the transpose.

Step 6: Compute U (left singular vectors)

$$U = AV\Sigma^{-1}$$

- Multiply A × V
- Then divide each column by the corresponding singular value in Σ

Approximate U:

$$U \approx \begin{bmatrix} 0.293 & 0.707 & 0.5 & 0.415 \\ 0.707 & 0 & -0.707 & 0 \\ 0.5 & -0.707 & -0.5 & 0.415 \\ 0.293 & 0 & 0 & -0.707 \end{bmatrix}$$

- Columns of U = **left singular vectors**
- Now we have all three matrices **mathematically derived**.

Step 7: Verify SVD

Check if:

$$A \approx U \Sigma V^T$$

- Multiply U × Σ × V^T → reconstruct original matrix A (small numerical errors possible due to approximations).
- This is the **final SVD solution**.

Step 8: Use U × Σ for Word Vectors

- Each **row of U × Σ** → vector for word in reduced space
- Example (approximate):

Word	Vector
Dog	[0.707, 1.0, …]
Cat	[1.707, -0.5, …]
Apple	[0.5, -1.0, …]
Mango	[0.293, 0, …]

These vectors can now be used to measure **similarity** (e.g., cosine similarity) between words.

Example Insight:

•Words that often occur in similar contexts will have similar low-dimensional vectors.

•“cat” and “dog” will end up close together.

- **Pros:**

•Captures global statistical structure.

•Semantics emerge from factorization.

- **Cons:**

•Expensive for large vocabularies.

•Not good for rare/new words.

•Word2Vec later became more efficient by using neural nets instead of huge matrices.

How Do These Numbers Come About?

- They are **not chosen manually**.
- They are **learned automatically** by a model (like Word2Vec, GloVe, FastText, BERT).
- Training uses a **large text corpus** (e.g., Wikipedia, Google News).
- The model adjusts the numbers (weights of a neural net or a factorized matrix) so that words appearing in **similar contexts** end up with **similar vectors**.

Why Only These Numbers?

1. Dimensionality (length of vector)

1. We choose a dimension size, e.g., 100, 200, or 300.
2. This is a **hyperparameter** decided by the user (not the algorithm).
3. Example: Word2Vec often uses 300 dimensions.

2. Values inside the vector

1. Start with random small numbers.
2. During training, numbers get updated (via gradient descent / optimization).
3. Eventually, the numbers settle into patterns that **encode meaning**.